

Tecnología Digital VI — Segundo Parcial

Modelo de Examen 3

Universidad Torcuato Di Tella

Duración: 2 horas

Nombre: _____

Parte 1: Verdadero o Falso (2 pts c/u)

Justifique brevemente las respuestas falsas.

- a) Las RNNs no pueden procesar secuencias de longitud variable, por lo que siempre se requiere padding y truncamiento a un largo fijo.
 - b) En el mecanismo de atención de Bahdanau, el vector de contexto c_t es un promedio ponderado de todos los hidden states del encoder, donde los pesos dependen del paso actual del decoder.
 - c) La representación Bag-of-Words preserva el orden de las palabras en un documento.
 - d) En el Inception Module (GoogLeNet), se aplican convoluciones de distintos tamaños en paralelo y se concatenan las salidas.
 - e) Early Stopping consiste en detener el entrenamiento cuando la loss de entrenamiento deja de bajar.
 - f) El modelo GPT es bidireccional: al generar texto, cada token puede “ver” tanto los tokens anteriores como los posteriores.
-

Parte 2: Multiple Choice (3 pts c/u)

Marque la opción correcta. Solo una es válida.

- 1) ¿Cuál de las siguientes NO es una ventaja del pooling en CNNs?
 - A) Reduce la dimensionalidad espacial.
 - B) Aumenta el campo receptivo.
 - C) Aprende nuevos filtros para detectar patrones.
 - D) Da robustez ante traslaciones del objeto.
- 2) ¿Qué problema resuelve **Leaky ReLU** respecto a ReLU estándar?

- A) El problema de vanishing gradients en funciones acotadas.
 - B) El problema de dying ReLU, permitiendo un gradiente pequeño para valores negativos.
 - C) La falta de derivabilidad en $x = 0$.
 - D) El costo computacional excesivo de ReLU.
- 3) En Word2Vec con arquitectura **CBOW**, ¿cuál es el input y cuál es el output?
- A) Input: palabra central. Output: palabras de contexto.
 - B) Input: palabras de contexto. Output: palabra central.
 - C) Input: toda la oración. Output: vector de embedding.
 - D) Input: one-hot de la palabra. Output: one-hot de la siguiente palabra.
- 4) ¿Cuál de las siguientes afirmaciones sobre **Autoencoders** es correcta?
- A) Requieren etiquetas supervisadas para entrenarse.
 - B) El encoder comprime la entrada a una representación latente y el decoder intenta reconstruir la entrada original.
 - C) Solo pueden usarse para clasificación de imágenes.
 - D) La loss function mide la diferencia entre la representación latente y el input.
- 5) ¿Qué componente del Transformer cumple un rol análogo a las skip connections de ResNet?
- A) Multi-Head Attention.
 - B) Feed-Forward Network.
 - C) Add & Norm (conexión residual + Layer Normalization).
 - D) Positional Encoding.
-

Parte 3: Desarrollo (10 pts c/u)

1) Funciones de activación y sus problemas.

- a) Explique el problema de *Vanishing Gradients* en redes profundas que usan Sigmoid o Tanh. ¿Por qué la multiplicación repetida de derivadas pequeñas impide el aprendizaje en las primeras capas?
- b) ¿Cómo soluciona ReLU este problema? ¿Qué nuevo problema introduce (*Dying ReLU*)?
- c) Nombre dos funciones de activación que mitigan el Dying ReLU y explique brevemente cómo lo logran.

2) Diseño de red para XOR.

Diseñe una red neuronal con una capa oculta de 2 neuronas (activación Heaviside/Step) y una neurona de salida que compute la función XOR. Especifique:

- a) Los pesos y biases de cada conexión.
- b) Verifique su diseño mostrando el forward pass para las 4 combinaciones posibles: $(0, 0)$, $(0, 1)$, $(1, 0)$, $(1, 1)$.

Hint: piense en XOR como la combinación de dos funciones linealmente separables.

3) Object Detection: R-CNN vs YOLO.

- a) Describa brevemente el pipeline de R-CNN (las 4 etapas principales).
- b) ¿Cuál es la diferencia fundamental entre R-CNN (two-stage) y YOLO (one-stage)?
- c) Explique qué es Non-Max Suppression y por qué es necesario en YOLO.
- d) Si YOLO usa una grilla de 7×7 con $B = 2$ bounding boxes y $C = 20$ clases, ¿cuántos valores tiene el tensor de salida?

Parte 4: Interpretación de Código (15 pts)

Analice el siguiente código de PyTorch y responda las preguntas.

```

1  import torch
2  import torch.nn as nn
3
4  class SentimentLSTM(nn.Module):
5      def __init__(self, vocab_size, embed_dim, hidden_dim, output_dim):
6          super().__init__()
7          self.embedding = nn.Embedding(vocab_size, embed_dim)
8          self.lstm = nn.LSTM(embed_dim, hidden_dim,
9                               batch_first=True,
10                               bidirectional=True)
11          self.fc = nn.Linear(hidden_dim * 2, output_dim)
12          self.sigmoid = nn.Sigmoid()
13
14      def forward(self, x):
15          embedded = self.embedding(x)
16          output, (hidden, cell) = self.lstm(embedded)
17          # Concatenar los hidden states de ambas direcciones
18          hidden_cat = torch.cat((hidden[0], hidden[1]), dim=1)
19          out = self.fc(hidden_cat)
20          return self.sigmoid(out)
21
22  modelo = SentimentLSTM(vocab_size=10000,
23                          embed_dim=100,
24                          hidden_dim=256,
25                          output_dim=1)
26  criterio = nn.BCELoss()
```

- a) ¿Qué tipo de problema resuelve esta red? Justifique basándose en la dimensión de salida y la loss function.

- b) ¿Qué hace la capa `nn.Embedding`? ¿Por qué es preferible a usar One-Hot Encoding directamente como entrada?
- c) ¿Por qué `self.fc` recibe `hidden_dim * 2` como entrada? ¿Qué tiene que ver con el parámetro `bidirectional=True`?
- d) ¿Qué información contiene la variable `output` vs. la variable `hidden` que devuelve la LSTM?
- e) ¿Qué rol cumple la variable `cell` que devuelve la LSTM? ¿Se usa en este modelo?
- f) ¿Qué limitación tiene esta arquitectura (basada en LSTM) respecto a un Transformer para procesar secuencias largas?